

بِسْمِ خُدا

دانشگاه صنعتی خواجه نصیرالدین طوسی

گزارشکار پروژه ی دوم ریاضیات مهندسی
(فشرده سازی تصویر با تبدیل فوریه دوبعدی و بررسی
ضرایب فرکانسی)

استاد: دکتر خادم

گروه: آبتین احمدی (40317503)، علیرضا کهنه چیان (40324493)

مقدمه

ما در این پروژه ابتدا یک عکس پیش فرض نرم افزار متلب را با استفاده از تبدیل فوری به حوزه فوری منتقل کرده، فرکانس های بالای آن را با یک فیلتر Low-Pass حذف میکنیم تا حجم داده ها را کاهش دهیم، و سپس دوباره با عکس تبدیل فوری آن را به صورت تصویر در می آوریم تا ببینیم تغییر ضرایب فرکانسی در کیفیت عکس چه تاثیری خواهد داشت.

توضیحات کد

در ابتدا ما عکسی پیش فرض را وارد کد میکنیم، این عکس به علت ابعاد بالا در محاسبات (256×256)، کد ما را دچار مشکل خواهد کرد، به طوری که به علت $O(N^4)$ بودن تبدیل فوری دستی، کد در یک لوپ ابدی

```
%% --- Task 1: Image Preparation ---
% Load image
img = imread('cameraman.tif');
% Resize to 32x32 for feasible MANUAL calculation time (O(N^4) complexity)
% If you use original size, the manual loops will take forever!
target_size = [32, 32];
img_small = imresize(img, target_size);
img_double = double(img_small);
[M, N] = size(img_double);

figure('Name', 'Task 1: Original Image');
imshow(uint8(img_double));
title(['Original Image Resized (', num2str(M), 'x', num2str(N), ')']);
```



خواهد ماند! در نتیجه ابعاد

تصویر را با دستور `imresize`

کوچک میکنیم. سپس ابعاد آن را

به صورت آرایه ای دو بعدی

تعریف میکنیم که هر خانه قابل

استناد باشد. تصویر به صورت مقابل

خواهد بود (تسک 1):

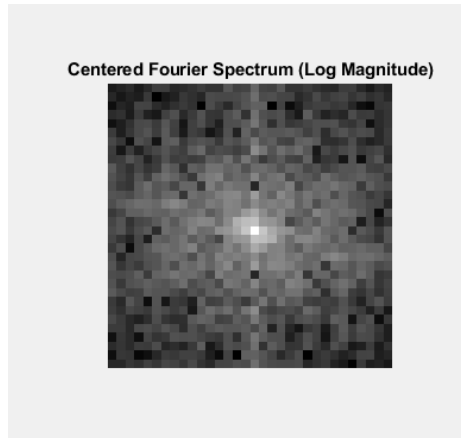
حال این تصویر را با دستور های `fft2` که نشان دهنده تبدیل فوری دو بعدی است، به حوزه فوری منتقل کرده و با دستور

fftshift، فرکانس های صفر را از گوشه های ماتریس به مرکز آن منتقل میکنیم. (تسک 2)

```
%% --- Task 2: Built-in FFT2 ---
% Calculate FFT using built-in function
F_builtin = fft2(img_double);

% Shift zero frequency to center
F_centered = fftshift(F_builtin);

% Visualize Log Magnitude Spectrum
figure('Name', 'Task 2: Spectrum');
imshow(log(1 + abs(F_centered)), []);
title('Centered Fourier Spectrum (Log Magnitude)');
```



حال برای تسک 3، با توجه به فرمول داده شده

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \cdot e^{-2\pi i \left(\frac{ux}{M} + \frac{vy}{N} \right)}$$

```
%% --- Task 3: Manual Forward DFT Implementation ---
% Formula: F(u,v) = sum_x sum_y f(x,y) * e^(-j*2*pi*(ux/M + vy/N))
F_manual = zeros(M, N);

for u = 0:M-1
    for v = 0:N-1
        sum_val = 0;
        for x = 0:M-1
            for y = 0:N-1
                % Note: Matlab indices start at 1, so use x+1, y+1
                term = img_double(x+1, y+1) * exp(-1j * 2 * pi * ((u*x)/M + (v*y)/N));
                sum_val = sum_val + term;
            end
        end
        F_manual(u+1, v+1) = sum_val;
    end
end

% Verification check
diff_norm = norm(abs(F_builtin - F_manual));
fprintf('Difference between Built-in and Manual Forward DFT: %e\n', diff_norm);
```

آن را در کد پیاده سازی میکنیم:

برای اینکه ببینیم محاسبات دستی و نرم افزار چه مقدار فرق میکنند، از دستور انتهای این بخش استفاده میکنیم که خروجی زیر را به ما میدهد:

Difference between Built-in and Manual Forward DFT:

7.660219e-10

عدد داده عملاً صفر می باشد که نشان دهنده دقت کار ما است.

```
%% --- Task 4 & 5: Compression and Reconstruction for different percentages ---
```

```
% Define the percentages to test
percentages = [0.05, 0.10, 0.50]; % 5%, 10%, and 50%

% Create a figure to display all results side-by-side
figure('Name', 'Task 4 & 5: Compression Results Comparison');

% Display the original image in the first position
subplot(1, length(percentages) + 1, 1);
imshow(uint8(img_double));
title('Original');

% Loop through each percentage value
for i = 1:length(percentages)
    p = percentages(i);
```

در قسمت بعدی دو تسک 4 و 5 باهم انجام میشوند. در تسک 4 از ما خواسته شده تا با استفاده از یک Low Pass Filter، فرکانس

های بالا را حذف کنیم، برای این کار باید یک ماسک دایره ای اطراف ماتریس تعریف کنیم، به صورتی که با تغییر دادن درصد موردنظر، شعاع

```
% Loop through each percentage value
for i = 1:length(percentages)
    p = percentages(i);

    % --- Task 4: Create Mask and Apply it ---

    % Create a circular mask based on the current percentage 'p'
    [Y, X] = meshgrid(1:N, 1:M);
    cx = round(N/2);
    cy = round(M/2);
    radius_sq = (X - cx).^2 + (Y - cy).^2;

    % Calculate the radius that keeps the top 'p'% of coefficients
    keep_radius = sqrt((p * M * N) / pi);

    mask = radius_sq <= keep_radius^2;

    % Apply mask to the CENTERED spectrum to zero out high frequencies
    F_compressed_centered = F_centered .* mask;
```

این دایره تغییر میکند و در نتیجه مقادیر داخل دایره اضافه یا کم میشوند. شعاع دایره به صورت مقابل تعریف میشود:

همانطور که مشاهده میشود،

شعاع با درصد نگه داشتن ضرایب

مرکزی، رابطه مستقیم دارد و میتوان به صورت تئوری تایید کرد که با درصد بزرگتر، دایره ی بزرگتری داریم، در نتیجه فرکانس های بیشتری در دایره ماسک قرار میگیرند و در نهایت تصویر حاصل، کیفیت بهتری خواهد داشت.

حال برای تسک 5، به علت کند بودن روش دستی، آن را پیاده سازی نمیکنیم. چرا که باید هر خانه بررسی شود که آیا مقداری برابر با صفر دارد یا خیر. برای باز تولید تصویر، باید ابتدا شیفت مرکزی را برگردانیم،

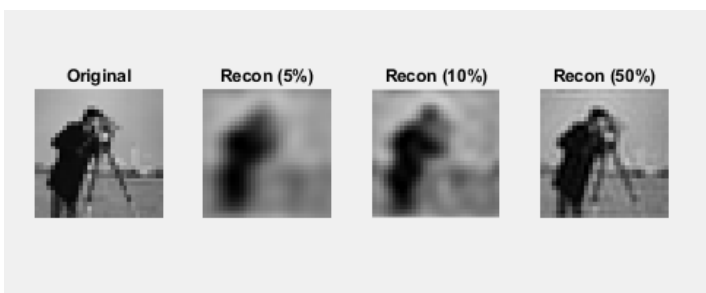
```
% --- Task 5: Inverse Transform ---

% Un-shift the compressed spectrum
F_compressed = ifftshift(F_compressed_centered);

% Reconstruct the image using built-in inverse FFT
img_recon_builtin = ifft2(F_compressed);
|
% Get the real part (imaginary part should be negligible)
img_recon_builtin_real = real(img_recon_builtin);

% --- Display the result ---
subplot(1, length(percentages) + 1, i + 1);
imshow(uint8(img_recon_builtin_real));
title(['Recon (' num2str(p * 100) '%)']);

fprintf('Compression with p = %.2f done.\n', p);
end
```



عکس تبدیل فوریه را پیاده سازی کنیم و قسمت موهومی را حذف کنیم. در نتیجه روش نرم افزاری آن به شکل زیر خواهد بود:

و نتیجه تصاویری که مشاهده خواهیم کرد به شکل زیر هستند:

همانطور که مشاهده میشود، با بالا رفتن درصد حفظ ضرایب مرکزی، وضوح تصویر بالا می رود که تئوری ما را که بر حسب شعاع دایره ماسک بود، تایید می کند.

برای تسک 7، ما میتوانیم برای مقایسه نتیجه نرم افزار و محاسبات دستی، مقدار $F(0,0)$ را در هر دو روش بدست آوریم. برای روش نرم افزار کافیهست $f(x,y)$ ها را به ازای $u = 0$ و $v = 0$ بدست آوریم، که این شرایط نرم نمایی ما را برابر یک میکند، در نتیجه تنها باید جمع $f(x,y)$ ها را محاسبه کنیم؛ و برای روش دستی تنها نیاز داریم به خانه $F_manual(1,1)$ که بالا تر در تسک 3 آن را تعریف کردیم، دست پیدا کنیم.

```
%% --- Task 7 & 8: Analysis & Manual Check of F(0,0) ---

% F(0,0) Manual Calculation (Average Intensity / Sum of pixels)
% Formula: sum of all f(x,y) for u=0, v=0 (exp term becomes 1)
F_00_manual_calc = sum(sum(img_double));
F_00_from_fft = F_manual(1,1); % Matlab index (1,1) is (0,0)

fprintf('\n--- Task 7: F(0,0) Check ---\n');
fprintf('Sum of all pixels (Manual F(0,0)): %.4f\n', F_00_manual_calc);
fprintf('Value from FFT code F(0,0): %.4f\n', F_00_from_fft);
|
```

خروجی این بخش از کد به صورت زیر خواهد بود که به ما نشان میدهد

--- Task 7: $F(0,0)$ Check ---

Sum of all pixels (Manual $F(0,0)$): 121585.0000

Value from FFT code $F(0,0)$: 121585.0000

که این دو روش هر دو یک

پاسخ مشخص دارند.

همچنین سوالات تسک 8 در طول تسک های دیگر، پاسخ داده شده است.